Python Engineering - EX23

WEEK 04 | CLASS 15 - Exceptions & Debugging | Safe File Loading

Goal: learn to detect, interpret, and handle predictable failures while reading engineering data. We will test a valid CSV, a missing file, and corrupted data.

> The goal is not to make errors disappear. The goal is to know **what failed, where, why, and what the program should do next**.

Exact path

```
01_python_bootcamp/
`-- week_04_oop_files_testing/
    `-- class_15_exceptions_debugging/
        |-- notes/
        `-- exercises/
            `-- ex23_safe_file_loading/
                |-- file_io.py
                |-- main.py
                `-- data/
                    |-- valid_trajectory.csv
                    `-- corrupted_trajectory.csv
```

Three scenarios

Case	What happens	Expected exception
Valid file	Correct file and numeric columns	None
Missing file	Path points to a file that does not exist	FileNotFoundError
Corrupted file	A row violates the expected format	ValueError

Reference files

```
valid_trajectory.csv
h_m,v_m_s,t_s
100.000000,20.000000,0.000000
102.000000,19.019000,0.100000
103.901900,18.038000,0.200000

corrupted_trajectory.csv
h_m,v_m_s,t_s
100.000000,20.000000,0.000000
102.000000,NOT_A_NUMBER,0.100000
103.901900,18.038000,0.200000
```

BLOCK A - Load CSV without hiding failures

```
import numpy as np

def load_trajectory_csv(file_path):

    rows = []

    with open(file_path, 'r', encoding='utf-8') as file:
        header = file.readline().strip()

        if header != 'h_m,v_m_s,t_s':
            raise ValueError('Invalid CSV header')

        for line_number, line in enumerate(file, start=2):
            clean_line = line.strip()
            if clean_line == '':
                continue

            values = clean_line.split(',')

            if len(values) != 3:
                raise ValueError(
                    f'Line {line_number}: expected 3 columns'
                )

            h = float(values[0])
            v = float(values[1])
            t = float(values[2])
            rows.append([h, v, t])

    return np.array(rows)
```

The loader does not silently repair bad input. If the file contract is broken, it produces a clear exception.

BLOCK B - raise ValueError(...)

`raise` deliberately creates an exception when our own validation rule fails.

```
if len(values) != 3:
    raise ValueError(
        f'Line {line_number}: expected 3 columns'
    )
```

BLOCK C - enumerate(..., start=2)

```
for line_number, line in enumerate(file, start=2):
    ...
```

The first data row is physical file line 2 because line 1 is the header. Useful error messages include that location.

BLOCK D - Handle exceptions in main.py

```
from pathlib import Path
from file_io import load_trajectory_csv

def test_file(file_path):

    print(f'\nTesting: {file_path.name}')

    try:
        trajectory = load_trajectory_csv(file_path)

    except FileNotFoundError as error:
        print('FILE ERROR')
        print(type(error).__name__)
        print(error)

    except ValueError as error:
        print('DATA ERROR')
        print(type(error).__name__)
        print(error)

    else:
        print('LOAD OK')
        print(f'Shape: {trajectory.shape}')
        print(trajectory)

    finally:
        print('Test finished')
```

BLOCK E - try / except / else / finally flow

```
try
 |
 +-- success --------------------> else
 +-- FileNotFoundError ----------> matching except
 +-- ValueError -----------------> matching except
 |
 v
finally runs in every case
```

try: potentially failing code. **except:** response to a specific failure. **else:** only after success. **finally:** always runs.

BLOCK F - Build the three paths

```
base_dir = Path(__file__).resolve().parent
data_dir = base_dir / 'data'

valid_file = data_dir / 'valid_trajectory.csv'
missing_file = data_dir / 'missing_trajectory.csv'
corrupted_file = data_dir / 'corrupted_trajectory.csv'
```

Do not create missing_trajectory.csv.

BLOCK G - Run all three cases

```
files_to_test = [
    valid_file,
    missing_file,
    corrupted_file
]

for file_path in files_to_test:
    test_file(file_path)
```

Expected behavior

```
valid -> LOAD OK -> finally
missing -> FileNotFoundError except -> finally
corrupted -> ValueError except -> finally
```

BLOCK H - Deliberate VS Code debugging

```
Suggested breakpoints:
1. values = clean_line.split(',')
2. v = float(values[1])
3. except ValueError as error:
```

Inspect `line_number`, `clean_line`, `values`, `values[1]`, and its type.

BLOCK I - Read a traceback

Temporarily disable the ValueError handler and run the corrupted file. Read the traceback, then restore the handler.

```
Look for:
1. Exception type
2. Exception message
3. Last relevant line in your own code
```

A practical habit is to start near the bottom of a traceback and move upward until you reach the relevant line in your code.

FINAL VALIDATION

```
Explain in comments:
- What is an exception?
- Exception vs syntax error?
- What belongs inside try?
- Why separate FileNotFoundError and ValueError?
- What does 'as error' give us?
- When do else and finally execute?
- What does raise do?
- Why raise ValueError for invalid structure?
- Why avoid a broad except Exception here?
- What does enumerate(..., start=2) provide?
- Why include line numbers in messages?
- What is a traceback?
- Which part should you inspect first?
- Why is silently returning an empty array dangerous?
- Handling an error vs hiding an error?
```

Completion criterion

EX23 is complete when you can deliberately trigger all three cases, predict the executed branch, explain each exception, and locate the problematic value with the debugger or traceback.